

# Widgets de Flutter

Catálogo práctico de widgets de Flutter, alineado con Material 3: fundamentos, atomización, layout, navegación con GoRouter, formularios, listas, animaciones y un arsenal de +150 widgets de referencia rápida. Sin state management.

ENFOQUE

**Widgets puros ·  
Material 3**

STACK

**Flutter + Dart**

NIVEL

**Principiante a  
Intermedio**

TIEMPO ESTIMADO

**6-10 horas**

Widgets

Material 3

GoRouter

Flutter

# Contenido

---

11 capítulos · Catálogo de widgets de Flutter (Material 3)

PARTE 1 · FUNDAMENTOS

011. Fundamentos de Widgets
022. Widgets Básicos y Atomización

PARTE 2 · LAYOUT, INTERACCIÓN Y LISTAS

033. Layout y Navegación
044. Interacción y Formularios
055. Listas y Scroll

PARTE 3 · ESTADO Y ANIMACIÓN

066. Datos, Estados y Ciclo de Vida
077. Animaciones

PARTE 4 · COMPOSICIÓN Y CALIDAD

088. Estrategias de Composición
099. Patrones de Renderización
1010. Rendimiento y Buenas Prácticas

PARTE 5 · REFERENCIA RÁPIDA

1111. Arsenal Completo de Widgets

# 1. Fundamentos de Widgets

Versión objetivo: **Flutter 3.44+ / 3.47 (estable)** — Material 3 por defecto. La documentación oficial de referencia está en [docs.flutter.dev](https://docs.flutter.dev).

## Qué es un widget

En Flutter, **todo es un widget**. No hay separación entre "vistas", "layout" y "controles" como en otros frameworks. Un widget es una **descripción inmutable de una parte de la UI**.

```
// Esto ES la UI. No hay XML, no hay templates separados.  
const Text('Hola mundo');  
const Icon(Icons.star);
```

Como los widgets son inmutables, Flutter no los modifica: los **reemplaza** por otros. Es un modelo **declarativo**: describes *cómo debería verse* la UI para un estado dado, y Flutter se encarga de llevarla a ese estado.

## Los tres árboles (widget / element / render)

Cada widget que escribes participa en tres árboles paralelos:

1. **Widget tree** — la configuración que escribes (barato de crear y descartar).
2. **Element tree** — las instancias mutables que gestionan el ciclo de vida y que Flutter *reutiliza* al reconstruir (la clave del rendimiento).
3. **Render tree** — los `RenderObject` que calculan layout y pintan en pantalla (`RenderBox`, `RenderParagraph`, etc.).

```
MaterialApp — Widget —> Element —> RenderView  
└ Scaffold —> Element —> RenderBox (layout + paint)
```

No necesitas manipular `Element` ni `RenderObject` directamente, pero entender que existen te ayuda a comprender por qué ciertos patrones (como las `key`) funcionan y por qué reconstruir un widget es barato mientras el `Element` se conserve.

# StatelessWidget vs StatefulWidget

## StatelessWidget

No tiene estado mutable. Se construye y solo se reconstruye si su configuración (propiedades) cambia.

```
class MiAvatar extends StatelessWidget {
  final String nombre;
  final double tamaño;

  const MiAvatar({super.key, required this.nombre, this.tamaño = 48});

  @override
  Widget build(BuildContext context) {
    return CircleAvatar(
      radius: tamaño / 2,
      child: Text(nombre[0].toUpperCase()),
    );
  }
}
```

Usa `StatelessWidget` cuando: el widget depende solo de sus propiedades y del contexto (`Theme`, `MediaQuery`, etc.).

## StatefulWidget

Tiene estado mutable que puede cambiar durante la vida del widget. El estado vive en la clase `State`, **no** en el widget (el widget sigue siendo inmutable).

```
class ContadorWidget extends StatefulWidget {
  const ContadorWidget({super.key});

  @override
  State<ContadorWidget> createState() => _ContadorWidgetState();
}

class _ContadorWidgetState extends State<ContadorWidget> {
  int _contador = 0;

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('$_contador'),
        FilledButton(
          onPressed: () => setState(() => _contador++),
          child: const Text('Incrementar'),
        ),
      ],
    );
  }
}
```

Cada vez que llamas `setState()`, Flutter programa una **reconstrucción** del subárbol de este widget. El patrón es siempre: cambia el dato → `setState` → `build` se llama de nuevo.

## El árbol de widgets

Los widgets se anidan formando un árbol:

```
MaterialApp
├── Scaffold
│   ├── AppBar
│   └── body: Column
│       ├── Text
│       ├── SizedBox
│       └── FilledButton
```

Cada `child` o `children` crea un nivel en el árbol. Este anidamiento es normal y esperado — Flutter lo maneja de forma eficiente.

## BuildContext

`BuildContext` es el "lugar" de un widget en el árbol: un **handle** al `Element`. Se usa para:

- Acceder al tema: `Theme.of(context)`
- Navegar: `Navigator.of(context)`
- Leer dependencias del árbol: `MediaQuery.sizeOf(context)`, `context.read<T>()`, `context.watch<T>()` (estos dos últimos vienen de los paquetes `provider` / `flutter_bloc`, no del núcleo)

```
@override
Widget build(BuildContext context) {
  // context te da acceso a todo el árbol hacia arriba
  final theme = Theme.of(context);
  final screenWidth = MediaQuery.sizeOf(context).width;

  return Container(
    color: theme.colorScheme.primary,
    width: screenWidth * 0.9,
    child: const Text('Responsive'),
  );
}
```

`MediaQuery.sizeOf(context)` solo reconstruye el widget cuando cambia el **tamaño** de la pantalla. `MediaQuery.of(context).size` (vieja forma) reconstruía ante *cualquier* cambio de `MediaQuery` (tema, padding, etc.). Desde Flutter 3.35 también existen `MediaQuery.widthOf` y `MediaQuery.heightOf`. **Prefiere los accesores `xOf`**. `context.read<T>()` y `context.watch<T>()` pertenecen al módulo 16-BLOC-CUBIT. Aquí los mencionamos porque aparecerán en ejemplos de la guía como adelante.

## const y las reconstrucciones

Siempre que puedas, declara widgets como `const`:

```
// MAL: se crea un nuevo widget en cada build
child: Text('Hola');

// BIEN: se reusa la misma instancia constante
child: const Text('Hola');

// MEJOR: constructores const en tus propios widgets
class MiWidget extends StatelessWidget {
  const MiWidget({super.key});
  // ...
}
```

Los widgets `const` permiten a Flutter **saltarse por completo** la reconstrucción de ese subárbol: como la instancia nunca cambia, el `Element` se conserva sin necesidad de reconciliar.

## Keys

Las keys ayudan a Flutter a identificar widgets cuando el árbol cambia de posición o de tipo. Sin una key, Flutter compara por *posición* y tipo; con key, por *identidad*.

```
ListView(children: [
  // Sin key, al reordenar Flutter no sabe qué elemento es cuál
  // y puede conservar el estado equivocado (ej. texto escrito)
  TodoItem(tarea: tareas[0]),
  TodoItem(tarea: tareas[1]),
])

// Con ValueKey, Flutter preserva el estado correcto al reordenar
ListView(children: [
  TodoItem(key: ValueKey(tareas[0].id), tarea: tareas[0]),
  TodoItem(key: ValueKey(tareas[1].id), tarea: tareas[1]),
])
```

| Key                                       | Cuándo usarla                                                                                                                                          |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ValueKey&lt;T&gt;</code><br>(valor) | El identificador es un valor ( <code>id</code> , <code>String</code> , <code>int</code> ) — la más común                                               |
| <code>ObjectKey(objeto)</code>            | El identificador es un objeto completo sin <code>==</code> propio                                                                                      |
| <code>UniqueKey()</code>                  | Solo cuando quieras <i>forzar</i> que el widget se re-cree siempre                                                                                     |
| <code>GlobalKey&lt;...&gt;()</code>       | Para acceder al <code>State</code> o <code>RenderObject</code> desde fuera (ej. <code>Form</code> , <code>AnimatedList</code> ) — úsala con moderación |

## Regla de oro

Pregúntate siempre: ¿este widget necesita cambiar con el tiempo?

- No → `StatelessWidget`
- Sí, pero solo por datos externos → `StatelessWidget` + `Stream` / `Future` / `InheritedWidget`

- Sí, y necesita estado interno efímero → `StatefulWidget`

---

## Referencias

---

- [Flutter | Introduction to widgets](#) — Arquitectura de los 3 árboles y el modelo declarativo
  - [Flutter | Understanding constraints](#) — Cómo el layout fluye por el árbol (widget/element/render)
  - [Flutter | API reference — Widget](#) — Clase base de todos los widgets
  - [Flutter | API reference — BuildContext](#) — API del contexto (dependOn/read/watch, maybeOf)
  - [Flutter | Keys](#) — Cuándo y por qué usar keys
- 

## Lo que sigue

---

En el próximo capítulo veremos los widgets básicos de UI, cómo se agrupan en componentes reutilizables y el patrón de atomización.

## 2. Widgets Básicos y Atomización

---

### Text

---

El widget más fundamental para mostrar texto.

```
const Text(  
  'Hola Mundo',  
  style: TextStyle(  
    fontSize: 20,  
    fontWeight: FontWeight.bold,  
    color: Colors.blue,  
  ),  
  textAlign: TextAlign.center,  
  maxLines: 2,  
  overflow: TextOverflow.ellipsis,  
);
```

Propiedades clave: `style`, `textAlign`, `maxLines`, `overflow`, `softWrap`.

Para estilos heredables usa `DefaultTextStyle` o el `textTheme` del tema (ver capítulo 8):

```
Text(  
  'Título de la tarjeta',  
  style: Theme.of(context).textTheme.titleMedium,  
);
```

### Icon

---

Íconos del set Material Design.

```
const Icon(Icons.favorite, color: Colors.red, size: 32);
```

Usa siempre `Icons.*`. Evita imágenes para íconos simples. Nota: Material Icons tiene su propia librería de fuentes; si usas iconos personalizados de Google Fonts en runtime, se necesita el paquete `google_fonts`. En la mayoría de los casos `Icons.*` es suficiente.

## Imagen

```
// Desde assets (declarada en pubspec.yaml)
Image.asset('assets/imagenes/logo.png');

// Desde red
Image.network('https://ejemplo.com/foto.jpg');

// Con errorBuilder y control de decodificación
Image.network(
  'https://ejemplo.com/foto.jpg',
  errorBuilder: (context, error, stackTrace) =>
    const Icon(Icons.broken_image),
  filterQuality: FilterQuality.medium,
);

// Con placeholder y caché (requiere cached_network_image)
CachedNetworkImage(
  imageUrl: 'https://ejemplo.com/foto.jpg',
  placeholder: (_, __) => const CircularProgressIndicator(),
  errorWidget: (_, __, ___) => const Icon(Icons.error),
);
```

Para rendimiento, siempre declara dimensiones (`width` / `height`) o `cacheWidth` / `cacheHeight` en imágenes de red (ver capítulo 10).

## SizedBox y Container

Contenedores para espaciado y decoración.

```
const SizedBox(height: 16); // Espaciador simple
const SizedBox(width: 100, height: 100); // Caja de tamaño fijo

Container(
  width: 100,
  height: 100,
  decoration: BoxDecoration(
    color: Colors.blue,
    borderRadius: BorderRadius.circular(8),
    boxShadow: [
      BoxShadow(color: Colors.black26, blurRadius: 4),
    ],
  ),
  child: const Text('Dentro'),
);
```

Prefiere `SizedBox` sobre `Container` cuando solo necesites espacio. `Container` es más pesado. Para casos simples usa los widgets especializados: `ColoredBox` (solo color), `DecoratedBox` (solo decoración), `Padding` (solo espaciado) — evitan las restricciones que `Container` impone.

## Padding, Margin, Center, Align

Widgets de alineación y espaciado puro.

```
const Padding(  
  padding: EdgeInsets.all(16),  
  child: Text('Con padding'),  
);  
  
const Center(child: Text('Centrado'));  
  
Align(  
  alignment: Alignment.bottomRight,  
  child: const Text('Esquina'),  
);
```

## Botones (Material 3)

Cada versión de Material Design tiene su familia de botones. Con Material 3, `ElevatedButton` se considera legado; usa `FilledButton`.

```
FilledButton(  
  onPressed: () {},  
  child: const Text('Relleno'),  
);  
FilledButton.tonal(  
  onPressed: () {},  
  child: const Text('Tonal'),  
);  
OutlinedButton(  
  onPressed: () {},  
  child: const Text('Outline'),  
);  
TextButton(  
  onPressed: () {},  
  child: const Text('Texto'),  
);
```

Para botones con ícono:

```
FilledButton.icon(  
  onPressed: () {},  
  icon: const Icon(Icons.add),  
  label: const Text('Agregar'),  
);
```

Los botones se deshabilitan pasando `onPressed: null`. Para agrupar opciones exclusivas usa `SegmentedButton` (ver capítulo 4).

## Chip

Etiquetas compactas para mostrar metadata y filtrar.

```
const Chip(label: Text('Flutter'));  
  
InputChip(  
  label: const Text('Dart'),  
  onSelected: (selected) {},  
  avatar: const Icon(Icons.code),  
);  
  
FilterChip(  
  label: const Text('Activo'),  
  selected: _filtroActivo,  
  onSelected: (selected) => setState(() => _filtroActivo = selected),  
);  
  
ChoiceChip(  
  label: const Text('Pequeño'),  
  selected: _tamaño == 'peq',  
  onSelected: (_) => setState(() => _tamaño = 'peq'),  
);
```

## Card

Contenedor elevado con bordes redondeados, ideal para listas. Con Material 3 existen variantes `filled` y `outlined`.

```
Card(  
  child: Padding(  
    padding: const EdgeInsets.all(16),  
    child: Column(  
      crossAxisAlignment: CrossAxisAlignment.start,  
      children: [  
        const Text('Título', style: TextStyle(fontSize: 18)),  
        const SizedBox(height: 8),  
        Text('Descripción de la tarjeta'),  
      ],  
    ),  
  ),  
);  
  
// Variantes M3: menos elevación, más sutiles  
Card.filled(child: /* ... */);  
Card.outlined(child: /* ... */);
```

## Badge

Indicador superpuesto (notificaciones, contadores).

```
Badge(  
  label: const Text('3'),  
  child: const Icon(Icons.notifications),  
);  
  
// Sin child: badge independiente sobre el borde superior  
Badge.count(count: 99, child: const Icon(Icons.shopping_cart));
```

## Patrón de atomización

Los widgets se agrupan en una jerarquía de atomic design:

|            |                                         |
|------------|-----------------------------------------|
| Átomos     | → Text, Icon, Chip, Botones, SizedBox   |
| Moléculas  | → Card, ListTile, AppBar, NavigationBar |
| Organismos | → Formulario, Lista, Drawer, Scaffold   |
| Plantillas | → Página específica con layout          |
| Páginas    | → Ruta completa con Providers           |

Crea tus propios widgets atómicos para mantener consistencia:

```
class AppText extends StatelessWidget {  
  final String texto;  
  final TextStyle? style;  
  
  const AppText(this.texto, {super.key, this.style});  
  
  @override  
  Widget build(BuildContext context) {  
    return Text(  
      texto,  
      style: (Theme.of(context).textTheme.bodyLarge ?? const TextStyle())  
        .merge(style),  
    );  
  }  
}
```

## Referencias

- [Flutter | Widget catalog](#) — Catálogo completo de widgets por categoría
- [Flutter | Material 3](#) — Guía de Material 3 en Flutter
- [Flutter | Text](#) — API del widget Text
- [Flutter | Card](#) — API de Card y variantes M3

# Lo que sigue

---

Ahora que conoces los bloques básicos, el siguiente capítulo cubre cómo organizarlos en layouts y sistemas de navegación.

## 3. Layout y Navegación

### Row y Column

Los dos layouts fundamentales.

```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceEvenly, // eje horizontal  
  crossAxisAlignment: CrossAxisAlignment.center,    // eje vertical  
  children: const [  
    Icon(Icons.star, color: Colors.amber),  
    Text('4.5'),  
    Text('(120 reseñas)'),  
  ],  
);  
  
Column(  
  mainAxisAlignment: MainAxisAlignment.start,  
  crossAxisAlignment: CrossAxisAlignment.stretch,  
  children: [  
    const Text('Título'),  
    const SizedBox(height: 8),  
    const Text('Contenido'),  
  ],  
);
```

| Propiedad                       | Row        | Column     |
|---------------------------------|------------|------------|
| <code>mainAxisAlignment</code>  | horizontal | vertical   |
| <code>crossAxisAlignment</code> | vertical   | horizontal |
| <code>mainAxisSize</code>       | ancho      | alto       |

### Flex y Expanded

Distribución proporcional del espacio.

```
Row(  
  children: [  
    Expanded(flex: 2, child: Container(color: Colors.red)),  
    Expanded(flex: 1, child: Container(color: Colors.blue)),  
    // La primera ocupa 2/3, la segunda 1/3  
  ],  
);
```

Usa `Flexible` cuando quieras que un hijo pueda ser más pequeño que su flex:

```
Row(  
  children: [  
    Flexible(child: Text(largoTexto, overflow: TextOverflow.ellipsis)),  
    const Icon(Icons.arrow_forward),  
  ],  
);
```

## Stack

---

Superposición de widgets. Ideal para badges, capas, overlays.

```
Stack(  
  children: [  
    Image.network('https://ejemplo.com/fondo.jpg'),  
    const Positioned(  
      bottom: 16,  
      left: 16,  
      child: Text('Texto sobre imagen',  
        style: TextStyle(color: Colors.white, fontSize: 24)),  
    ),  
    const Positioned(  
      top: 8,  
      right: 8,  
      child: CircleAvatar(child: Text('A')),  
    ),  
  ],  
);
```

## Scaffold

---

Estructura base de una pantalla. Con Material 3, la barra inferior recomendada es `NavigationBar`.

```

Scaffold(
  appBar: AppBar(
    title: const Text('Mi App'),
    actions: [
      IconButton(icon: const Icon(Icons.search), onPressed: () {}),
    ],
  ),
  body: const Center(child: Text('Contenido')),
  floatingActionButton: FloatingActionButton(
    onPressed: () {},
    child: const Icon(Icons.add),
  ),
  bottomNavigationBar: NavigationBar(
    selectedIndex: _tabIndex,
    onDestinationSelected: (index) => setState(() => _tabIndex = index),
    destinations: const [
      NavigationDestination(icon: Icon(Icons.home), label: 'Inicio'),
      NavigationDestination(icon: Icon(Icons.search), label: 'Buscar'),
      NavigationDestination(icon: Icon(Icons.person), label: 'Perfil'),
    ],
  ),
  drawer: NavigationDrawer(
    children: const [
      DrawerHeader(child: Text('Menú')),
      ListTile(leading: Icon(Icons.settings), title: Text('Ajustes')),
    ],
  ),
);

```

`BottomNavigationBar` y `Drawer` siguen funcionando pero son estilos de Material 2. En Material 3 usa `NavigationBar` y `NavigationDrawer`.

## Navigator 1.0 (push/pop)

Navegación imperativa clásica.

```

// Ir a otra pantalla
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => const DetallePage(id: 42),
  ),
);

// Regresar
Navigator.pop(context);

// Recibir resultado
final resultado = await Navigator.push<String>(
  context,
  MaterialPageRoute(builder: (_) => const SeleccionPage()),
);

```

## Navegación con GoRouter (recomendada)

Navegación declarativa basada en rutas. Es el estándar actual para apps reales y lo usaremos a lo largo de la guía.

```
// pubspec.yaml: go_router: ^17.5.0

final router = GoRouter(
  initialLocation: '/',
  routes: [
    GoRoute(
      path: '/',
      builder: (_, __) => const HomePage(),
    ),
    GoRoute(
      path: '/productos',
      builder: (_, __) => const ProductosPage(),
      routes: [
        GoRoute(
          path: ':id',
          builder: (_, state) => DetalleProductoPage(
            id: state.pathParameters['id']!,
          ),
        ),
      ],
    ),
  ],
);
```

Uso en `MaterialApp.router`:

```
MaterialApp.router(
  routerConfig: router,
  // ...
);
```

Navegación con GoRouter:

```
// context.go reemplaza la ruta actual
context.go('/productos');

// context.push agrega al historial
context.push('/productos/42');

// context.pop regresa
context.pop();

// Parámetros extra (query)
context.push('/productos/42?source=home');
```

Con `go_router` también puedes proteger rutas con `redirect` (por ejemplo, redirigir a login si no hay sesión). Veremos esto en profundidad en los módulos de arquitectura.

## SafeArea

---

Asegura que el contenido no se superponga con áreas del sistema (notch, barra de estado, etc.).

```
SafeArea(  
  child: Column(  
    children: [  
      // Este contenido respeta las áreas seguras  
    ],  
  ),  
);
```

## MediaQuery

---

Información del tamaño y orientación de la pantalla. Usa los accesorios `xOf` para reconstruir solo cuando cambie lo que te interesa.

```
@override  
Widget build(BuildContext context) {  
  final ancho = MediaQuery.sizeOf(context).width; // solo cambia con el tamaño  
  final alto = MediaQuery.heightOf(context); // solo cambia con el alto  
  final isMobile = ancho < 600;  
  
  return isMobile ? const _MobileLayout() : const _TabletLayout();  
}
```

`MediaQuery.of(context).size` (forma anterior) reconstruía el widget ante *cualquier* cambio de MediaQuery (tema, padding, etc.). Desde Flutter 3.35 prefiere `MediaQuery.sizeOf` / `widthOf` / `heightOf`.

## LayoutBuilder

---

Adaptación al espacio disponible del padre (no de la pantalla).

```
LayoutBuilder(  
  builder: (context, constraints) {  
    if (constraints.maxWidth < 400) {  
      return const _VerticalLayout();  
    }  
    return const _HorizontalLayout();  
  },  
);
```

## PopScope (reemplaza WillPopScope)

---

Para interceptar el botón de retroceso del sistema. `WillPopScope` está deprecado desde Flutter 3.24.

```
PopScope(  
  canPop: false, // bloquea el pop  
  onPopInvokedWithResult: (didPop, result) {  
    if (!didPop) {  
      // Confirmar salida antes de permitir el pop  
      _mostrarConfirmacion();  
    }  
  },  
  child: const Scaffold(body: Text('Edición pendiente')),  
);
```

En versiones futuras, `onPopInvokedWithResult` reemplaza por completo a `onPopInvoked` (deprecado en 3.29).

---

## Referencias

---

- [Flutter | Layouts](#) — Guía de layouts en Flutter
  - [Flutter | Understanding constraints](#) — Cómo funcionan las restricciones
  - [Flutter | Navigation and routing](#) — Navegación declarativa e imperativa
  - [go\\_router | pub.dev](#) — Documentación de go\_router
  - [Flutter | PopScope](#) — Interceptar el retroceso del sistema
- 

## Lo que sigue

---

Pasamos a interacción del usuario: gestos, formularios y manejo de entrada.

## 4. Interacción y Formularios

---

### GestureDetector e InkWell

---

Captura de gestos táctiles.

```
GestureDetector(  
  onTap: () => print('Tap'),  
  onDoubleTap: () => print('Doble tap'),  
  onLongPress: () => print('Presión larga'),  
  onHorizontalDragEnd: (_) => print('Swipe'),  
  child: const Text('Tocable'),  
);
```

**InkWell** agrega feedback visual Material (splash):

```
InkWell(  
  onTap: () {},  
  child: Container(  
    padding: const EdgeInsets.all(16),  
    child: const Text('Con efecto splash'),  
  ),  
);
```

### TextField y TextFormField

---

Entrada de texto libre.

```

TextField(
  controller: _controller,
  decoration: InputDecoration(
    labelText: 'Nombre',
    hintText: 'Ingresa tu nombre',
    prefixIcon: const Icon(Icons.person),
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(8),
    ),
    suffixIcon: _controller.text.isNotEmpty
      ? IconButton(
          icon: const Icon(Icons.clear),
          onPressed: () => _controller.clear(),
        )
      : null,
  ),
  keyboardType: TextInputType.text,
  textInputAction: TextInputAction.next,
  obscureText: _oculto,
  onChanged: (value) => setState(() {}),
);

```

Controladores deben ser dispuestos:

```

class _FormState extends State<FormWidget> {
  final _nombreCtrl = TextEditingController();
  final _emailCtrl = TextEditingController();

  @override
  void dispose() {
    _nombreCtrl.dispose();
    _emailCtrl.dispose();
    super.dispose();
  }
}

```

Extras útiles: `autofillHints: [AutofillHints.email]` para autocompletado nativo, `onTapOutside` para ocultar teclado, y `enabled: false` para deshabilitar sin perder el valor.

## Form con validación

```
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: Column(
    children: [
      TextFormField(
        decoration: const InputDecoration(labelText: 'Email'),
        validator: (value) {
          if (value == null || value.isEmpty) return 'Campo requerido';
          if (!value.contains('@')) return 'Email inválido';
          return null;
        },
      ),
      const SizedBox(height: 16),
      FilledButton(
        onPressed: () {
          if (_formKey.currentState!.validate()) {
            // Procesar formulario
          }
        },
        child: const Text('Enviar'),
      ),
    ],
  ),
);
```

`_formKey.currentState!.validate()` valida todos los campos y `save()` guarda los valores en los `TextFormField` (si definiste `onSaved`).

## DropDownButtonFormField

Selección de una opción entre varias. Desde Flutter 3.35 usa `initialValue` en lugar de `value`.

```
String? _rol;

DropDownButtonFormField<String>(
  initialValue: _rol,
  decoration: const InputDecoration(labelText: 'Rol'),
  items: const [
    DropdownMenuItem(value: 'admin', child: Text('Admin')),
    DropdownMenuItem(value: 'user', child: Text('Usuario')),
    DropdownMenuItem(value: 'guest', child: Text('Invitado')),
  ],
  onChanged: (value) => setState(() => _rol = value),
);
```

## DropDownMenu (M3)

Alternativa moderna a `DropDownButton` con campo de búsqueda y estilo M3.

```

DropdownMenu<String>(
  label: const Text('Rol'),
  initialSelection: _rol,
  onSelected: (value) => setState(() => _rol = value),
  dropdownMenuEntries: const [
    DropdownMenuEntry(value: 'admin', label: 'Admin'),
    DropdownMenuEntry(value: 'user', label: 'Usuario'),
    DropdownMenuEntry(value: 'guest', label: 'Invitado'),
  ],
);

```

## Checkbox, Switch, Radio (RadioGroup)

Controles binarios y de selección única.

```

// Checkbox
Checkbox(
  value: _aceptaTerminos,
  onChanged: (value) => setState(() => _aceptaTerminos = value!),
);

// Switch – desde 3.35: activeThumbColor (activeColor está deprecado)
Switch(
  value: _notificaciones,
  activeThumbColor: Colors.green,
  onChanged: (value) => setState(() => _notificaciones = value),
);

// Radio – desde 3.32/3.35 el grupo se gestiona con RadioGroup
RadioGroup<String>(
  groupValue: _genero,
  onChanged: (value) => setState(() => _genero = value),
  child: const Column(
    children: [
      Radio(value: 'masculino'),
      Radio(value: 'femenino'),
      Radio(value: 'otro'),
    ],
  ),
);

```

En `Radio` los parámetros `groupValue` y `onChanged` están deprecados desde 3.32: el estado del grupo lo maneja `RadioGroup`. Cada `Radio` solo declara su `value`.

## SegmentedButton

Selección exclusiva estilizada (M3), ideal para filtros con pocas opciones.

```
SegmentedButton<String>(  
  segments: const [  
    ButtonSegment(value: 'dia', label: Text('Día'), icon: Icon(Icons.wb_sunny)),  
    ButtonSegment(value: 'mes', label: Text('Mes')),  
    ButtonSegment(value: 'anio', label: Text('Año')),  
  ],  
  selected: {_periodo},  
  onSelectionChanged: (selection) =>  
    setState(() => _periodo = selection.first),  
);
```

## DatePicker y TimePicker

```
Future<void> _seleccionarFecha() async {  
  final fecha = await showDatePicker(  
    context: context,  
    initialDate: DateTime.now(),  
    firstDate: DateTime(2020),  
    lastDate: DateTime(2030),  
  );  
  if (fecha != null) {  
    setState(() => _fecha = fecha);  
  }  
}  
  
Future<void> _seleccionarHora() async {  
  final hora = await showTimePicker(  
    context: context,  
    initialTime: TimeOfDay.now(),  
  );  
  if (hora != null) {  
    setState(() => _hora = hora);  
  }  
}
```

## Slider y RangeSlider

---

```
// Slider simple
Slider(
  value: _volumen,
  min: 0,
  max: 100,
  divisions: 10,
  label: '${_volumen.round()}',
  onChanged: (value) => setState(() => _volumen = value),
);

// RangeSlider para rangos
RangeSlider(
  values: _rango,
  min: 0,
  max: 1000,
  divisions: 10,
  labels: TextRange(
    start: '${_rango.start.round()}',
    end: '${_rango.end.round()}',
  ),
  onChanged: (values) => setState(() => _rango = values),
);
```

## Patrón de formulario con estado local (widgets puros)

---

Con `ValueNotifier` + `ValueListenableBuilder` puedes construir un formulario reactivo sin librerías externas. El botón se habilita/deshabilita según el estado.

```

class LoginForm extends StatefulWidget {
  const LoginForm({super.key});
  @override
  State<LoginForm> createState() => _LoginFormState();
}

class _LoginFormState extends State<LoginForm> {
  final _formKey = GlobalKey<FormState>();
  final _emailCtrl = TextEditingController();
  final _passCtrl = TextEditingController();
  final _enviando = ValueNotifier<bool>(false);

  @override
  void dispose() {
    _emailCtrl.dispose();
    _passCtrl.dispose();
    _enviando.dispose();
    super.dispose();
  }

  Future<void> _iniciarSesion() async {
    if (!(_formKey.currentState?.validate() ?? false)) return;
    _enviando.value = true;
    try {
      await autenticar(_emailCtrl.text, _passCtrl.text); // tu lógica
      if (mounted) context.go('/home');
    } finally {
      if (mounted) _enviando.value = false;
    }
  }

  @override
  Widget build(BuildContext context) {
    return Form(
      key: _formKey,
      child: Column(
        children: [
          TextFormField(
            controller: _emailCtrl,
            decoration: const InputDecoration(labelText: 'Email'),
            validator: (v) =>
              v?.contains('@') == true ? null : 'Email inválido',
          ),
          TextFormField(
            controller: _passCtrl,
            decoration: const InputDecoration(labelText: 'Contraseña'),
            obscureText: true,
            validator: (v) =>
              (v?.length ?? 0) >= 6 ? null : 'Mínimo 6 caracteres',
          ),
          ValueListenableBuilder<bool>(
            valueListenable: _enviando,
            builder: (context, enviando, child) {
              return FilledButton(
                onPressed: enviando ? null : _iniciarSesion,
                child: enviando
                  ? const SizedBox(
                      width: 20, height: 20,
                      child: CircularProgressIndicator(strokeWidth: 2),
                    )
                  : const Text('Iniciar sesión'),
              );
            },
          ),
        ],
      ),
    );
  }
}

```

```
    ),  
    ],  
  ),  
);  
}  
}
```

En el módulo 16 este mismo formulario se versiona con `BlocProvider` + `BlocListener` + `BlocBuilder`; aquí lo mantenemos en widgets puros porque el tema de esta guía es el catálogo de widgets. La estructura del árbol (`Form` → campos → botón) es idéntica en ambos casos.

---

## Referencias

- [Flutter | TextFields](#) — Catálogo de campos de texto
- [Flutter | Material 3 — Selection controls](#) — Checkbox, Switch, Radio en M3
- [Flutter | API — RadioGroup](#) — Grupo de radios con navegación por teclado
- [Flutter | API — SegmentedButton](#) — Botón segmentado M3
- [Flutter | Forms](#) — Guía de formularios

---

## Lo que sigue

El siguiente capítulo cubre listas y scroll: cómo mostrar colecciones de datos eficientemente.

## 5. Listas y Scroll

### ListView

El widget base para listas desplazables.

```
// Lista finita con hijos conocidos
ListView(
  children: const [
    ListTile(title: Text('Item 1')),
    ListTile(title: Text('Item 2')),
    ListTile(title: Text('Item 3')),
  ],
);

// Lista infinita (solo construye los visibles)
ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, index) {
    return ListTile(
      title: Text(items[index].nombre),
      subtitle: Text(items[index].descripcion),
      leading: CircleAvatar(child: Text('${index + 1}')),
      onTap: () => _onItemTap(items[index]),
    );
  },
);
```

### ListTile

Widget optimizado para filas de lista.

```
ListTile(
  leading: CircleAvatar(
    backgroundImage: NetworkImage(usuario.avatarUrl),
  ),
  title: Text(usuario.nombre),
  subtitle: Text(usuario.email),
  trailing: IconButton(
    icon: const Icon(Icons.more_vert),
    onPressed: () {},
  ),
  onTap: () => _abrirPerfil(usuario),
  selected: usuario.id == seleccionadoId,
);
```

## ListView.separated

---

Con separadores entre ítems.

```
ListView.separated(  
  itemCount: productos.length,  
  separatorBuilder: (_, __) => const Divider(height: 1),  
  itemBuilder: (context, index) {  
    return ListTile(title: Text(productos[index].nombre));  
  },  
);
```

## GridView

---

Para layouts de cuadrícula.

```
// Con número fijo de columnas  
GridView.count(  
  crossAxisCount: 2,  
  mainAxisSpacing: 8,  
  crossAxisSpacing: 8,  
  padding: const EdgeInsets.all(8),  
  children: [  
    for (final item in items)  
      Card(child: Center(child: Text(item.nombre))),  
  ],  
);  
  
// Con tamaño dinámico  
GridView.builder(  
  gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(  
    crossAxisCount: 2,  
    childAspectRatio: 0.8,  
  ),  
  itemCount: items.length,  
  itemBuilder: (context, index) => _ItemCard(item: items[index]),  
);
```

## CustomScrollView + Slivers

---

Scroll avanzado con efectos combinados.

```

CustomScrollView(
  slivers: [
    SliverAppBar(
      title: const Text('Mi App'),
      pinned: true,
      expandedHeight: 200,
      flexibleSpace: FlexibleSpaceBar(
        background: Image.network('https://ejemplo.com/banner.jpg',
          fit: BoxFit.cover),
      ),
    ),
    SliverToBoxAdapter(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Text('Sección especial'),
      ),
    ),
    SliverList(
      delegate: SliverChildBuilderDelegate(
        (context, index) => ListTile(title: Text('Item $index')),
        childCount: 50,
      ),
    ),
    SliverGrid(
      gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: 3,
      ),
      delegate: SliverChildBuilderDelegate(
        (context, index) => Card(child: Center(child: Text('$index'))),
        childCount: 12,
      ),
    ),
  ],
);

```

Nota: en la vista scrollable, `cacheExtent` está deprecado desde Flutter 3.44 (los viewports ahora calculan su caché automáticamente). No lo declares.

## CarouselView (3.35+)

Carrusel de tarjetas con scroll por páginas, construido sobre la familia de scroll.

```

CarouselView(
  itemSnapping: true,
  itemExtent: 280,
  children: [
    for (final banner in banners)
      _BannerCard(banner: banner),
  ],
);

```

## Pull to refresh

---

```
Future<void> _recargar() async {
  // Carga tu data; RefreshIndicator muestra el spinner mientras espera
  final data = await cargarProductos();
  if (mounted) setState(() => _productos = data);
}

RefreshIndicator(
  onRefresh: _recargar,
  child: ListView.builder(
    itemCount: _productos.length,
    itemBuilder: (context, index) => ListTile(
      title: Text(_productos[index].nombre),
    ),
  ),
);
```

## Infinite scroll (paginación)

---

Con un `StatefulWidget` + `ScrollController`, sin librerías externas.

```

class _ProductosListaState extends State<ProductosLista> {
  final _scrollCtrl = ScrollController();
  final _items = <Producto>[];
  bool _isLoadingMore = false;

  @override
  void initState() {
    super.initState();
    _scrollCtrl.addListener(_onScroll);
    _cargarPagina();
  }

  @override
  void dispose() {
    _scrollCtrl.removeListener(_onScroll);
    _scrollCtrl.dispose();
    super.dispose();
  }

  void _onScroll() {
    if (_scrollCtrl.position.pixels >=
        _scrollCtrl.position.maxScrollExtent - 200) {
      _cargarPagina();
    }
  }

  Future<void> _cargarPagina() async {
    if (_isLoadingMore) return;
    _isLoadingMore = true;
    final nuevos = await cargarProductos(
      offset: _items.length,
    );
    if (mounted) {
      setState(() {
        _items.addAll(nuevos);
        _isLoadingMore = false;
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      controller: _scrollCtrl,
      itemCount: _items.length + (_isLoadingMore ? 1 : 0),
      itemBuilder: (context, index) {
        if (index >= _items.length) {
          return const Center(
            child: Padding(
              padding: EdgeInsets.all(16),
              child: CircularProgressIndicator(),
            ),
          );
        }
        return ListTile(title: Text(_items[index].nombre));
      },
    );
  }
}

```

## ScrollController utilidades

---

```
// Scroll a posición específica
_scrollCtrl.animateTo(
  0, // tope
  duration: const Duration(milliseconds: 300),
  curve: Curves.easeInOut,
);

// Detectar si está en el tope
bool get estaEnTope => _scrollCtrl.position.pixels <= 0;

// Scroll infinito con ScrollNotification
NotificationListener<ScrollNotification>(
  onNotification: (notification) {
    if (notification is ScrollEndNotification &&
        _scrollCtrl.position.pixels >=
        _scrollCtrl.position.maxScrollExtent) {
      _cargarPagina();
    }
    return false;
  },
  child: ListView.builder(/* ... */),
);
```

## SingleChildScrollView

---

Para contenido que no es lista pero puede desbordar.

```
SingleChildScrollView(
  padding: const EdgeInsets.all(16),
  child: Column(
    children: [
      _FormularioCompleto(),
      const SizedBox(height: 24),
      _TerminosYCondiciones(),
    ],
  ),
);
```

## AnimatedList

---

Lista con animaciones de inserción/eliminación.

```

final _listKey = GlobalKey<AnimatedListState>();

AnimatedList(
  key: _listKey,
  initialItemCount: items.length,
  itemBuilder: (context, index, animation) {
    return SizeTransition(
      sizeFactor: animation,
      child: ListTile(title: Text(items[index])),
    );
  },
);

// Insertar
_listKey.currentState!.insertItem(
  index,
  duration: const Duration(milliseconds: 300),
);

// Eliminar
_listKey.currentState!.removeItem(
  index,
  (context, animation) => SizeTransition(
    sizeFactor: animation,
    child: ListTile(title: Text(items[index])),
  ),
);

```

Existe la variante sliver `SliverAnimatedList` para usarla dentro de un `CustomScrollView`.

## ReorderableListView

Lista reordenable por drag & drop.

```

ReorderableListView(
  onReorder: (oldIndex, newIndex) {
    setState(() {
      if (newIndex > oldIndex) newIndex--;
      final item = _items.removeAt(oldIndex);
      _items.insert(newIndex, item);
    });
  },
  children: [
    for (final item in _items)
      ListTile(
        key: ValueKey(item.id), // key obligatoria
        title: Text(item.nombre),
      ),
  ],
);

```

## Referencias

---

- [Flutter | ListView](#) — API de ListView y variantes
  - [Flutter | Slivers](#) — Guía de slivers y CustomScrollView
  - [Flutter | CarouselView](#) — Carrusel de contenido (3.35+)
  - [Flutter | AnimatedList](#) — Listas con animaciones
- 

## Lo que sigue

---

El capítulo 6 cubre datos, estados y ciclo de vida: cómo fluye la información en una app Flutter.

## 6. Datos, Estados y Ciclo de Vida

---

### Flujo de datos en Flutter

---

Los datos viajan **hacia abajo** por el árbol de widgets (vía constructores o `InheritedWidget`). Las notificaciones (callbacks) viajan **hacia arriba**.

```
Datos    → Widget Padre → Widget Hijo  
Callback → Hijo → Padre
```

Este flujo unidireccional se conoce como *lifting state up*: el estado se eleva al ancestro común que lo necesita, y baja a los hijos que solo lo muestran.

## StatefulWidget ciclo de vida

```
class MiWidget extends StatefulWidget {
  const MiWidget({super.key});

  @override
  State<MiWidget> createState() => _MiWidgetState();
}

class _MiWidgetState extends State<MiWidget> {
  @override
  void initState() {
    super.initState();
    // 1. Inicializar controladores, suscribirse a streams, cargar datos
  }

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    // 2. Se llama después de initState.
    // También se llama cuando cambia un InheritedWidget usado en build.
  }

  @override
  void didUpdateWidget(covariant MiWidget oldWidget) {
    super.didUpdateWidget(oldWidget);
    // 3. Se llama cuando el padre reconstruye con nuevas configuraciones.
  }

  @override
  void dispose() {
    // 4. Liberar controladores, cancelar suscripciones.
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    // Se llama después de cada setState, didChangeDependencies,
    // didUpdateWidget.
    return const Text('Ciclo de vida');
  }
}
```

Orden en montaje: **constructor** → **initState** → **didChangeDependencies** → **build**

Orden en cambio de padre: **didUpdateWidget** → **build**

Orden en desmontaje: **dispose**

## FutureBuilder

Widget reactivo que construye UI basada en el estado de un **Future**.

```

FutureBuilder<List<Producto>>(
  future: _cargarProductos(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Center(child: CircularProgressIndicator());
    }

    if (snapshot.hasError) {
      return Center(
        child: Column(
          children: [
            Text('Error: ${snapshot.error}'),
            FilledButton(
              onPressed: () => setState(() {}), // reintentar
              child: const Text('Reintentar'),
            ),
          ],
        ),
      );
    }

    if (!snapshot.hasData || snapshot.data!.isEmpty) {
      return const Center(child: Text('Sin datos'));
    }

    return ListView.builder(
      itemCount: snapshot.data!.length,
      itemBuilder: (context, index) {
        return ListTile(title: Text(snapshot.data![index].nombre));
      },
    );
  },
);

```

Almacena el `Future` en una variable/estado si quieres que el reintento cree uno nuevo. Si lo creas directamente en `future:`, cada rebuild lanza una nueva petición.

## StreamBuilder

Widget reactivo que se actualiza con cada emisión de un `Stream`.

```

StreamBuilder<int>(  
  stream: contadorStream,  
  builder: (context, snapshot) {  
    if (snapshot.connectionState == ConnectionState.waiting) {  
      return const Text('Esperando...');  
    }  
  
    if (snapshot.hasError) {  
      return Text('Error: ${snapshot.error}');  
    }  
  
    return Text(  
      'Valor: ${snapshot.data}',  
      style: const TextStyle(fontSize: 32),  
    );  
  },  
);

```

Puedes pasar `initialData` para pintar algo antes de la primera emisión.

## ValueListenableBuilder y ListenableBuilder

Widgets reactivos para `ValueNotifier` y cualquier `Listenable` — estado local simple sin streams.

```

final _contador = ValueNotifier<int>(0);

ValueListenableBuilder<int>(  
  valueListenable: _contador,  
  builder: (context, value, child) {  
    return Column(  
      children: [  
        Text('$value', style: const TextStyle(fontSize: 32)),  
        FilledButton(  
          onPressed: () => _contador.value++,  
          child: const Text('+'),  
        ),  
      ],  
    );  
  },  
);

```

`ListenableBuilder` es la versión genérica (acepta `ChangeNotifier`, `Animation`, etc. sin desempacar valores):

```

ListenableBuilder(  
  listenable: _modelo, // un ChangeNotifier  
  builder: (context, child) => Text(_modelo.titulo),  
);

```

`ValueNotifier` + `ValueListenableBuilder` es ideal para estado local que no requiere BLoC.

`ChangeNotifier` + `ListenableBuilder` sirve cuando el estado es un objeto completo.

## TickerProvider

Necesario para animaciones. Provee el `Ticker` que sincroniza con el refresh rate.

```
class _AnimacionState extends State<AnimacionWidget>
  with SingleTickerProviderStateMixin {
  late final AnimationController _ctrl;

  @override
  void initState() {
    super.initState();
    _ctrl = AnimationController(
      vsync: this, // TickerProvider
      duration: const Duration(seconds: 1),
    );
  }

  @override
  void dispose() {
    _ctrl.dispose();
    super.dispose();
  }
}
```

Usa `TickerProviderStateMixin` (en vez de `SingleTickerProviderStateMixin`) si necesitas más de un controller.

## Patrones de carga de datos en initState

```
class _PaginaState extends State<PaginaWidget> {
  @override
  void initState() {
    super.initState();
    _cargar();
  }

  Future<void> _cargar() async {
    // Con Future: iniciar carga
    final data = await servicio.fetch();
    // setState seguro porque el widget está montado
    if (!mounted) return;
    setState(() => _data = data);
  }
}
```

Cuando tu capa de estado viva en otra parte (Cubit/BLoC, módulo 16), `initState` solo dispara la intención:

```
@override
void initState() {
  super.initState();
  // Adelanto del módulo 16: con BLoC se dispara un evento,
  // el cubit emite el estado y la UI reacciona.
  context.read<MiCubit>().cargarDatos();
}
```

## mounted y context.mounted

`mounted` indica si el `State` sigue en el árbol. Úsalo antes de tocar el estado después de un `await`.

```
@override
Widget build(BuildContext context) {
  return FilledButton(
    onPressed: () async {
      await Future.delayed(const Duration(seconds: 3));
      // Si el widget ya no está montado, no tocar el estado
      if (!mounted) return;
      setState(() { /* ... */ });
    },
    child: const Text('Click'),
  );
}
```

Desde Flutter 3.7 también existe `context.mounted`, útil en funciones que reciben un `BuildContext` sin `State` (por ejemplo, un método de una clase helper):

```
Future<void> irADetalle(BuildContext context) async {
  await obtenerAlgo();
  if (!context.mounted) return; // evita usar un context desmontado
  Navigator.of(context).push(...);
}
```

Diferencia: `mounted` (en el `State`) protege `setState`; `context.mounted` (en el `BuildContext`) protege operaciones que usan el context (navegación, `SnackBars`, `context.read`).



## Referencias

- Flutter | [StatefulWidget lifecycle](#) — Ciclo de vida y manejo de estado
- Flutter | [FutureBuilder](#) — API de FutureBuilder
- Flutter | [StreamBuilder](#) — API de StreamBuilder
- Flutter | [ListenableBuilder](#) — Builder reactivo genérico
- Flutter | [BuildContext.mounted](#) — Proteger usos asíncronos de context

## Lo que sigue

El siguiente capítulo cubre animaciones y cómo agregar movimiento a las interfaces.

## 7. Animaciones

### ImplicitlyAnimatedWidget

La familia de widgets con animaciones implícitas: solo cambias el target y Flutter interpola.

```
AnimatedContainer(  
  duration: const Duration(milliseconds: 300),  
  curve: Curves.easeInOut,  
  width: _expandido ? 200 : 100,  
  height: _expandido ? 200 : 100,  
  color: _expandido ? Colors.blue : Colors.red,  
  child: const Center(child: Text('Animar')),  
);  
  
AnimatedOpacity(  
  duration: const Duration(milliseconds: 300),  
  opacity: _visible ? 1.0 : 0.0,  
  child: const Text('Aparece/Desaparece'),  
);  
  
AnimatedPadding(  
  duration: const Duration(milliseconds: 300),  
  padding: _grande ? EdgeInsets.all(32) : EdgeInsets.all(8),  
  child: const Text('Con padding animado'),  
);  
  
AnimatedSwitcher(  
  duration: const Duration(milliseconds: 300),  
  child: Text(  
    '$_contador',  
    key: ValueKey(_contador), // clave única para la transición  
    style: const TextStyle(fontSize: 32),  
  ),  
);
```

Otros: `AnimatedPositioned`, `AnimatedAlign`, `AnimatedSize`, `AnimatedDefaultTextStyle`, `AnimatedSlide`, `AnimatedScale`, `AnimatedRotation`, `TweenAnimationBuilder`.

`TweenAnimationBuilder` anima desde un valor inicial hacia un valor final sin controller:

```
TweenAnimationBuilder<double>(  
  tween: Tween(begin: 0, end: _progreso),  
  duration: const Duration(milliseconds: 600),  
  builder: (context, value, child) => CircularProgressIndicator(  
    value: value,  
    backgroundColor: Colors.grey[300],  
  ),  
);
```

Los widgets implícitos aceptan `AnimationStyle` (desde 3.29) para personalizar duración/curva sin envolverlos.

## AnimationController

Control manual de animaciones.

```
class _MiAnimacionState extends State<MiAnimacion>
  with SingleTickerProviderStateMixin {
  late final AnimationController _ctrl;
  late final Animation<double> _anim;

  @override
  void initState() {
    super.initState();
    _ctrl = AnimationController(
      vsync: this,
      duration: const Duration(seconds: 2),
    );
    _anim = Tween<double>(begin: 0, end: 300).animate(_ctrl);
    _ctrl.forward();
  }

  @override
  void dispose() {
    _ctrl.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return AnimatedBuilder(
      animation: _anim,
      builder: (context, child) {
        return Container(
          width: _anim.value,
          height: _anim.value,
          color: Colors.blue,
          child: child,
        );
      },
      child: const Text('Crece'),
    );
  }
}
```

## CurvedAnimation

Aplica curvas de easing a la animación.

```
final curved = CurvedAnimation(
  parent: _ctrl,
  curve: Curves.easeOutBack,    // entrada
  reverseCurve: Curves.easeIn,  // salida
);
final _anim = Tween<double>(begin: 0, end: 1).animate(curved);
```

Curvas comunes: `easeIn`, `easeOut`, `easeInOut`, `bounceIn`, `easeOutBack`, `elasticOut`.

## AnimationStatusListener

Reaccionar a eventos del ciclo de la animación.

```
_ctrl.addStatusListener((status) {
  switch (status) {
    case AnimationStatus.completed:
      // Repetir o invertir
      _ctrl.reverse();
    case AnimationStatus.dismissed:
      _ctrl.forward();
    case AnimationStatus.forward:
    case AnimationStatus.reverse:
      // en progreso
  }
});
```

## Hero

Transición compartida entre pantallas.

```
// Pantalla origen
Hero(
  tag: 'avatar-{$usuario.id}',
  child: CircleAvatar(
    radius: 30,
    backgroundImage: NetworkImage(usuario.avatarUrl),
  ),
);

// Pantalla destino (mismo tag)
Hero(
  tag: 'avatar-{$usuario.id}',
  child: CircleAvatar(
    radius: 80, // tamaño diferente, anima automáticamente
    backgroundImage: NetworkImage(usuario.avatarUrl),
  ),
);
```

La transición ocurre automáticamente entre pantallas del mismo `Navigator` (también funciona con `context.push` de `go_router`).

## CustomPainter

---

Dibujo vectorial personalizado (gráficas, formas, backgrounds).

```
class MiPainter extends CustomPainter {
  final double progreso;

  MiPainter({required this.progreso});

  @override
  void paint(Canvas canvas, Size size) {
    final paint = Paint()
      ..color = Colors.blue
      ..style = PaintingStyle.fill;

    canvas.drawCircle(
      Offset(size.width / 2, size.height / 2),
      size.width / 2 * progreso,
      paint,
    );
  }

  @override
  bool shouldRepaint(covariant MiPainter old) => old.progreso != progreso;
}

// Uso
CustomPaint(
  size: const Size(100, 100),
  painter: MiPainter(progreso: 0.75),
);
```

## StaggeredAnimations

---

Múltiples animaciones encadenadas con diferentes delays (vía `Interval`).

```

class _StaggeredState extends State<StaggeredWidget>
  with SingleTickerProviderStateMixin {
  late final AnimationController _ctrl;
  late final Animation<double> _fade;
  late final Animation<Offset> _slide;

  @override
  void initState() {
    super.initState();
    _ctrl = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 1000),
    );
    _fade = Tween<double>(begin: 0, end: 1).animate(
      CurvedAnimation(parent: _ctrl, curve: const Interval(0.0, 0.5)),
    );
    _slide = Tween<Offset>(
      begin: const Offset(0, 0.5),
      end: Offset.zero,
    ).animate(
      CurvedAnimation(parent: _ctrl, curve: const Interval(0.3, 1.0)),
    );
    _ctrl.forward();
  }

  @override
  Widget build(BuildContext context) {
    return SlideTransition(
      position: _slide,
      child: FadeTransition(
        opacity: _fade,
        child: const Text('Animación escalonada'),
      ),
    );
  }
}

```

## Lottie / Rive (animaciones JSON)

```

// pubspec.yaml: lottie: ^3.1.0

Lottie.asset(
  'assets/animaciones/loading.json',
  width: 200,
  height: 200,
  animate: true,
  repeat: true,
);

// Con control manual
final _ctrl = LottieController();

Lottie.asset(
  'assets/animaciones/check.json',
  controller: _ctrl,
);
_ctrl.play();

```

`rive` (^0.13) ofrece animaciones interactivas con estados avanzados; ambas son buenas opciones para animaciones diseñadas en Figma/After Effects.

## Reglas para animaciones

---

- Prefiere animaciones implícitas ( `AnimatedContainer` , `AnimatedOpacity` ) para casos simples.
  - Usa `AnimationController` + `AnimatedBuilder` para control fino.
  - Mantén las animaciones en ~300ms para interacciones táctiles.
  - Usa `Curves.bounceIn` o `easeOutBack` para efectos llamativos controlados.
  - No animes: `opacity` + `size` simultáneamente (causa rebotes).
  - No uses `Opacity` en loops de animación: usa `FadeTransition` (ver capítulo 10).
- 



## Referencias

---

- [Flutter | Animations guide](#) — Guía oficial de animaciones
  - [Flutter | Implicit animations](#) — Widgets implícitos
  - [Flutter | Hero animations](#) — Transiciones compartidas
  - [lottie | pub.dev](#) — Animaciones de After Effects en Flutter
- 

## Lo que sigue

---

El siguiente capítulo cubre estrategias de composición de widgets para construir UIs complejas de forma mantenible.

## 8. Estrategias de Composición

### Composición > Herencia

En Flutter no se extienden widgets para modificarlos. Se **componen** conteniendo unos dentro de otros.

```
// MAL: heredar comportamiento
class MiBotonEspecial extends ElevatedButton {
  MiBotonEspecial({super.onPressed, super.child}); // frágil
}

// BIEN: componer con parámetros
class AccionBoton extends StatelessWidget {
  final String texto;
  final VoidCallback? onPressed;
  final IconData? icono;

  const AccionBoton({
    super.key,
    required this.texto,
    this.onPressed,
    this.icono,
  });

  @override
  Widget build(BuildContext context) {
    return FilledButton.icon(
      onPressed: onPressed,
      icon: icono != null ? Icon(icono) : const SizedBox.shrink(),
      label: Text(texto),
    );
  }
}
```

### Patrones slot (child y builder)

Para componentes reutilizables, expón un **child** (slot fijo) o un **builder** (slot que recibe contexto/estado):

```
// Slot con child: el padre decide el contenido
class PanelTitulo extends StatelessWidget {
  final String titulo;
  final Widget child; // slot

  const PanelTitulo({super.key, required this.titulo, required this.child});

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(titulo, style: Theme.of(context).textTheme.titleMedium),
            const SizedBox(height: 8),
            child,
          ],
        ),
      ),
    );
  }
}

// Slot con builder: el contenido depende del contexto local
class ItemSeleccionable<T> extends StatelessWidget {
  final T valor;
  final T seleccionado;
  final Widget Function(BuildContext, bool) builder;

  const ItemSeleccionable({
    super.key,
    required this.valor,
    required this.seleccionado,
    required this.builder,
  });

  @override
  Widget build(BuildContext context) {
    return builder(context, valor == seleccionado);
  }
}
```

## Widgets puros (sin estado)

Siempre que un widget pueda ser `StatelessWidget`, hazlo `const`. Esto permite que Flutter lo optimice.

```

class UserAvatar extends StatelessWidget {
  final String nombre;
  final String? fotoUrl;
  final double tamaño;

  const UserAvatar({
    super.key,
    required this.nombre,
    this.fotoUrl,
    this.tamaño = 48,
  });

  @override
  Widget build(BuildContext context) {
    return CircleAvatar(
      radius: tamaño / 2,
      backgroundImage: fotoUrl != null ? NetworkImage(fotoUrl!) : null,
      child: fotoUrl == null ? Text(nombre[0].toUpperCase()) : null,
    );
  }
}

```

## Widgets privados anidados

Divide widgets grandes en sub-widgets privados para mejorar legibilidad y rendimiento.

```

class PerfilPage extends StatelessWidget {
  const PerfilPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Perfil')),
      body: const _Body(),
    );
  }
}

class _Body extends StatelessWidget {
  const _Body();

  @override
  Widget build(BuildContext context) {
    return Column(
      children: const [
        _AvatarSection(),
        _InfoSection(),
        _AccionesSection(),
      ],
    );
  }
}

```

Cada sub-widget es un `const` que puede rebuildearse independientemente (ver capítulo 10).

## Separación de responsabilidades

|            |                                    |
|------------|------------------------------------|
| Página     | → Scaffold + AppBar + Providers    |
| Sección    | → Column/Row con agrupación lógica |
| Componente | → Widget reusable (atómico)        |

Cada widget debe tener **una** responsabilidad.

## Builder pattern para contextos

Cuando necesitas un `BuildContext` diferente (por ejemplo, dentro de un Scaffold para Snackbar):

```
// Builder crea un nuevo contexto en el árbol
Scaffold(
  body: Builder(
    builder: (context) {
      return FilledButton(
        onPressed: () {
          // Este contexto tiene acceso al Scaffold
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Hecho')),
          );
        },
        child: const Text('Mostrar SnackBar'),
      );
    },
  ),
);
```

## InheritedWidget (acceso implícito)

Es el mecanismo interno que usan `Theme`, `MediaQuery`, `InheritedNotifier`, etc. No lo uses directamente en la mayoría de los casos; entiende cómo funciona.

```
// Detrás de escena: BlocProvider (módulo 16) es un InheritedWidget
// context.read<T>() busca hacia arriba en el árbol sin suscribirse
// context.watch<T>() busca Y se suscribe a cambios

final theme = Theme.of(context);           // InheritedWidget
final ancho = MediaQuery.sizeOf(context).width; // accesorios xOf
final cubit = context.read<MiCubit>();      // BlocProvider → InheritedWidget
```

## Naming: sufijos de widgets

Usa sufijos consistentes para identificar el tipo de widget:

| Sufijo | Ejemplo           | Propósito                       |
|--------|-------------------|---------------------------------|
| Page   | LoginPage         | Ruta completa con Scaffold      |
| View   | ProfileView       | Contenido de una sección (body) |
| Card   | ProductCard       | Elemento de lista/tarjeta       |
| Tile   | ContactTile       | Fila de datos                   |
| Form   | RegisterForm      | Formulario con validación       |
| Dialog | ConfirmDialog     | Diálogo modal                   |
| Button | SocialButton      | Botón personalizado             |
| Row    | InfoRow           | Fila de datos horizontal        |
| Badge  | NotificationBadge | Indicador superpuesto           |

## Composición de layouts condicional

```
class ResponsiveLayout extends StatelessWidget {
  const ResponsiveLayout({super.key});

  @override
  Widget build(BuildContext context) {
    final width = MediaQuery.sizeOf(context).width;

    if (width < 600) {
      return const _MobileLayout();
    } else if (width < 1200) {
      return const _TabletLayout();
    }
    return const _DesktopLayout();
  }
}
```

Si la decisión depende del espacio del *padre* y no de la pantalla, usa `LayoutBuilder` (capítulo 3).

## Referencias

- Flutter | Understanding constraints — Cómo el layout fluye por el árbol
- Flutter | API — InheritedWidget — Acceso implícito a datos
- Flutter | Deep dive — keys — Composición y reconciliación
- Google | Atomic design — Origen del patrón de atomización

# Lo que sigue

---

Pasamos a patrones de renderización condicional y control de estado visual.

## 9. Patrones de Renderización

---

### Renderizado condicional

---

Tres patrones comunes para mostrar/ocultar widgets.

```
// 1. Operador ternario (más común)
child: estado == Estado.cargando
  ? const CircularProgressIndicator()
  : const Text('Listo'),

// 2. if-else en collection (Colecciones)
children: [
  const Text('Items'),
  if (items.isEmpty)
    const Text('No hay items')
  else
    for (final item in items)
      Text(item.nombre),
];

// 3. Visibility widget
Visibility(
  visible: _mostrar,
  child: const Text('Visible'),
  // Mantiene el espacio ocupado si se desea
  maintainState: false,
  maintainSize: false,
);
```

**Visibility** te permite elegir si el widget conserva espacio (**maintainSize**), estado (**maintainState**) o si se descarta del árbol por completo.

### Patrón Loaded / Error / Empty / Loading

---

Estado visual completo, modelado con un enum o clase sellada.

```

sealed class AsyncState<T> {}

class AsyncLoading<T> extends AsyncState<T> {}
class AsyncError<T> extends AsyncState<T> {
    final String message;
    AsyncError(this.message);
}
class AsyncData<T> extends AsyncState<T> {
    final T data;
    AsyncData(this.data);
}

class EstadoVisual<T> extends StatelessWidget {
    final AsyncState<T> estado;
    final Widget Function(T data) builder;

    const EstadoVisual({super.key, required this.estado, required this.builder});

    @override
    Widget build(BuildContext context) {
        return switch (estado) {
            AsyncLoading() => const Center(child: CircularProgressIndicator()),
            AsyncError(:final message) => Center(
                child: Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                        Text('Error: $message'),
                        FilledButton(
                            onPressed: () => {}, // reintentar (dispara nueva carga)
                            child: const Text('Reintentar'),
                        ),
                    ],
                ),
            ),
            AsyncData(:final data) when data is List && (data as List).isEmpty =>
                const Center(
                    child: Column(
                        mainAxisAlignment: MainAxisAlignment.center,
                        children: [
                            Icon(Icons.inbox, size: 64, color: Colors.grey),
                            SizedBox(height: 16),
                            Text('Sin datos'),
                        ],
                    ),
                ),
            AsyncData(:final data) => builder(data),
        };
    }
}

```

Uso (aquí con una clase de estado propia; en el módulo 16 la misma idea se aplica sobre `BlocBuilder`):

```

class ProductosBody extends StatelessWidget {
  final AsyncState<List<Producto>> estado;

  const ProductosBody({super.key, required this.estado});

  @override
  Widget build(BuildContext context) {
    return EstadoVisual(
      estado: estado,
      builder: (productos) => ListView.builder(
        itemCount: productos.length,
        itemBuilder: (context, index) => ListTile(
          title: Text(productos[index].nombre),
        ),
      ),
    );
  }
}

```

## Patrón Shimmer / Skeleton

Placeholder animado mientras carga.

```

// pubspec.yaml: shimmer: ^3.0.0

Shimmer.fromColors(
  baseColor: Colors.grey[300]!,
  highlightColor: Colors.grey[100]!,
  child: Column(
    children: [
      _SkeletonLine(width: double.infinity, height: 16),
      const SizedBox(height: 8),
      _SkeletonLine(width: 200, height: 16),
    ],
  ),
);

```

Sin dependencias externas:

```

Container(
  width: double.infinity,
  height: 16,
  decoration: BoxDecoration(
    color: Colors.grey[300],
    borderRadius: BorderRadius.circular(4),
  ),
);

```

## Patrón Sliver (scroll dinámico)

Combina listas, grids y headers en un solo scroll.

```

CustomScrollView(
  slivers: [
    SliverAppBar.large(
      title: const Text('Galería'),
      pinned: true,
    ),
    SliverToBoxAdapter(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Text('Últimos productos',
          style: Theme.of(context).textTheme.titleLarge),
      ),
    ),
    SliverGrid(
      gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: 2,
        mainAxisSpacing: 8,
        crossAxisSpacing: 8,
      ),
      delegate: SliverChildBuilderDelegate(
        (context, index) => _ProductCard(producto: productos[index]),
        childCount: productos.length,
      ),
    ),
  ],
);

```

## Patrón NestedScrollView

Header que colapsa mientras el contenido interno hace scroll.

```

NestedScrollView(
  headerSliverBuilder: (context, innerIsScrolled) => [
    SliverAppBar(
      title: const Text('Perfil'),
      expandedHeight: 200,
      flexibleSpace: FlexibleSpaceBar(
        background: Image.network('cover.jpg', fit: BoxFit.cover),
      ),
    ),
  ],
  SliverToBoxAdapter(
    child: _InfoTab(),
  ),
],
  body: TabBarView(
    children: [
      _PublicacionesTab(),
      _FotosTab(),
      _AmigosTab(),
    ],
  ),
);

```

## Patrón IndexedStack / PageView

---

Navegación entre vistas sin perder estado.

```
// IndexedStack: mantiene todas las vistas montadas
IndexedStack(
  index: _tabIndex,
  children: const [
    HomePage(),
    SearchPage(),
    ProfilePage(),
  ],
);

// PageView: swipe entre páginas
PageView(
  controller: _pageCtrl,
  children: const [
    OnboardingPage1(),
    OnboardingPage2(),
    OnboardingPage3(),
  ],
);
```

## Patrón Overlay / Popup

---

Superponer widgets sobre la UI actual.

```
// Modal bottom sheet
showModalBottomSheet(
  context: context,
  builder: (_) => Wrap(
    children: [
      ListTile(leading: const Icon(Icons.photo), title: const Text('Galería'), onTap: () {}),
      ListTile(leading: const Icon(Icons.camera), title: const Text('Cámara'), onTap: () {}),
    ],
  ),
);

// Tooltip
const Tooltip(
  message: 'Información adicional',
  child: Icon(Icons.info),
);

// PopupMenuButton
PopupMenuButton<String>(
  itemBuilder: (context) => [
    const PopupMenuItem(value: 'edit', child: Text('Editar')),
    const PopupMenuItem(value: 'delete', child: Text('Eliminar')),
  ],
  onSelected: (value) => print(value),
);
```

## Patrón Builder / Callback

---

Inversión de control: el hijo llama al padre.

```
class AccionLista extends StatelessWidget {
  final String itemId;
  final VoidCallback onEdit;
  final VoidCallback onDelete;

  const AccionLista({
    super.key,
    required this.itemId,
    required this.onEdit,
    required this.onDelete,
  });

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        IconButton(icon: const Icon(Icons.edit), onPressed: onEdit),
        IconButton(icon: const Icon(Icons.delete), onPressed: onDelete),
      ],
    );
  }
}
```

---

## Referencias

- [Flutter | Widget catalog](#) — Catálogo completo de widgets por categoría
- [Flutter | Slivers](#) — Slivers y scroll dinámico
- [Dart | Pattern matching](#) — `sealed` + `switch` para estados (3.10+)

---

## Lo que sigue

El capítulo 10 cubre rendimiento y buenas prácticas al trabajar con widgets.

## 10. Rendimiento y Buenas Prácticas

---

### const widgets

---

Es la optimización más importante y la más fácil.

```
// MAL
@override
Widget build(BuildContext context) {
  return Container(
    child: Text('Hola'), // nuevo objeto cada build
  );
}

// BIEN
@override
Widget build(BuildContext context) {
  return const SizedBox(
    width: 100, height: 100,
    child: Text('Hola'), // misma instancia siempre
  );
}
```

### Extraer sub-widgets

---

Dividir widgets grandes permite que Flutter reconstruya solo las partes necesarias.

```
// MAL: todo en el mismo build
@override
Widget build(BuildContext context) {
  return Column(
    children: [
      _Header(),      // cambia con el estado
      _Expensive(),   // NO cambia pero se reconstruye igual
      _Footer(),      // NO cambia pero se reconstruye igual
    ],
  );
}

// BIEN: Extraer _Expensive como StatelessWidget const
class _Expensive extends StatelessWidget {
  const _Expensive(); // ← const constructor

  @override
  Widget build(BuildContext context) {
    // Este widget NO se reconstruye si el padre cambia
    // porque es const y su padre no le pasa nuevos parámetros
    return Text('No cambio');
  }
}
```

## ListView.builder sobre ListView(children:[])

```
// MAL: construye todos los items aunque no sean visibles
ListView(children: items.map((i) => _Item(i)).toList());

// BIEN: solo construye los visibles
ListView.builder(
  itemCount: items.length,
  itemBuilder: (_, i) => _Item(items[i]),
);
```

## RepaintBoundary

Aísla partes del árbol para que no se repinten completas.

```
// Aísla el mapa para que no se repinte al hacer scroll
ListView(
  children: [
    RepaintBoundary(child: MapaWidget()),
    for (final item in items) _Item(item),
  ],
);

// Útil para: mapas, videos, animaciones aisladas, Canvas
```

## Evita reconstrucciones innecesarias

```
// MAL: nueva closure en cada build
@override
Widget build(BuildContext context) {
  return ChildWidget(
    callback: () { print('hola'); }, // nueva closure cada build
  );
}

// BIEN: método separado o clase
void _handleClick() => print('hola');

@override
Widget build(BuildContext context) {
  return ChildWidget(callback: _handleClick);
}
```

## AnimatedBuilder vs setState en animaciones

```
// MAL: setState en cada frame reconstruye todo el widget
@override
Widget build(BuildContext context) {
  return Container(
    width: _anim.value,
    height: _anim.value,
    child: const Text('Animado'),
  );
}

// BIEN: AnimatedBuilder solo reconstruye el subárbol necesario
AnimatedBuilder(
  animation: _anim,
  builder: (context, child) {
    return Container(
      width: _anim.value,
      height: _anim.value,
      child: child,
    );
  },
  child: const Text('Animado'), // no se reconstruye
);
```

## Keys para preservar estado

```
// Sin key: al reordenar, Flutter confunde los widgets
// Con key: Flutter sabe cuál mover
ListView.builder(
  itemBuilder: (_, i) => _Item(key: ValueKey(items[i].id), item: items[i]),
);
```

## Evitar Opacity con animaciones

---

```
// MAL: Opacity fuerza repintado del subárbol en cada cambio
Opacity(opacity: 0.5, child: const Text('Hola'));

// BIEN: AnimatedOpacity o FadeTransition (compositing)
FadeTransition(opacity: _anim, child: const Text('Hola'));
```

## Colores: usa withValues en vez de withOpacity

---

Desde Flutter 3.27, `Color.withOpacity` está deprecado. Usa `withValues(alpha:)`:

```
// MAL (deprecado desde 3.27)
Colors.black.withOpacity(0.5);

// BIEN
Colors.black.withValues(alpha: 0.5);
```

## Evitar Column/Row dentro de ListView.builder

---

```
// MAL: Column mide a sus hijos antes de renderizar
ListView.builder(
  itemBuilder: (_, i) => Column(children: [/* varios widgets */]),
);

// BIEN: si el item cabe en una sola línea, usa ListTile
ListView.builder(
  itemBuilder: (_, i) => ListTile(title: Text(items[i])),
);
```

## Imágenes: caché y dimensiones

```
// MAL: sin caché, tamaño original
Image.network('https://ejemplo.com/grande.jpg');

// BIEN: con caché y dimensiones controladas
CachedNetworkImage(
  imageUrl: 'https://ejemplo.com/grande.jpg',
  width: 200,
  height: 200,
  fit: BoxFit.cover,
  memCacheWidth: 400, // tamaño para memoria (2x por retina)
);

// Para múltiples imágenes de red, precarga:
Future.wait(
  urls.map((url) => precacheImage(NetworkImage(url), context)),
);
```

## Flutter DevTools

- **Rebuild Counts:** identifica widgets que se reconstruyen sin necesidad.
- **Track widget rebuilds:** muestra por qué se reconstruyó un widget.
- **Performance overlay:** muestra la velocidad de frames (target: 60fps o 120fps).
- **Memory & CPU:** detecta leaks y trabajo en el hilo de UI.

## Checklist de performance

- ☐ ¿Widgets hijos son `const` cuando no cambian?
- ☐ ¿ListView usa `.builder`?
- ☐ ¿Sub-widgets extraídos para evitar reconstrucciones masivas?
- ☐ ¿Imágenes tienen `width` / `height` y caché?
- ☐ ¿Animaciones con `AnimatedBuilder` en lugar de `setState`?
- ☐ ¿RepaintBoundary en mapas/videos/canvas?
- ☐ ¿Keys correctas en listas dinámicas?
- ☐ ¿Controladores (`TextEditingController`, `ScrollController`) dispuestos en `dispose`?



## Referencias

- [Flutter | Performance best practices](#) — Mejores prácticas de rendimiento

- [Flutter | DevTools](#) — Suite de herramientas de desarrollo
  - [Flutter | Widgets vs render objects](#) — Optimización del árbol de render
  - [Flutter | API — Color.withValues](#) — Reemplazo de withOpacity (3.27+)
- 

## Lo que sigue

---

El próximo capítulo es un arsenal de referencia rápida con todos los widgets vistos y más.

# 11. Arsenal Completo de Widgets

Referencia rápida de todos los widgets útiles para apps móviles. Cada entrada incluye una descripción de una línea y el uso típico. Alineado con **Material 3** y Flutter 3.44+.

## Texto y tipografía

| Widget                                     | Uso                                                            |
|--------------------------------------------|----------------------------------------------------------------|
| <code>Text</code>                          | Texto simple con estilo                                        |
| <code>RichText</code>                      | Texto con múltiples estilos en línea                           |
| <code>SelectableText</code>                | Texto seleccionable por el usuario                             |
| <code>DefaultTextStyle</code>              | Hereda estilo de texto a hijos                                 |
| <code>FittedBox</code>                     | Escala el hijo para que quepa en el espacio disponible         |
| <code>Wrap</code> de <code>TextSpan</code> | Composición de textos estilizados (en <code>Text.rich</code> ) |

## Íconos e imágenes

| Widget                          | Uso                                                                  |
|---------------------------------|----------------------------------------------------------------------|
| <code>Icon</code>               | Ícono Material Design                                                |
| <code>Image</code>              | Imagen desde asset, red, archivo o memoria                           |
| <code>CircleAvatar</code>       | Avatar circular con inicial o foto                                   |
| <code>FadeInImage</code>        | Imagen con fade al cargar                                            |
| <code>Placeholder</code>        | Espacio placeholder visual                                           |
| <code>CachedNetworkImage</code> | Imagen de red con caché (paquete <code>cached_network_image</code> ) |

## Botones

| Widget                          | Uso                         |
|---------------------------------|-----------------------------|
| <code>FilledButton</code>       | Botón primario M3 (relleno) |
| <code>FilledButton.tonal</code> | Botón secundario M3 (tonal) |

| Widget                                               | Uso                                          |
|------------------------------------------------------|----------------------------------------------|
| <code>OutlinedButton</code>                          | Botón con borde                              |
| <code>TextButton</code>                              | Botón de solo texto                          |
| <code>ElevatedButton</code>                          | Botón con sombra (legacy M2)                 |
| <code>IconButton</code>                              | Botón solo ícono                             |
| <code>IconButton.filled</code> / <code>.tonal</code> | Variantes M3 con fondo                       |
| <code>SegmentedButton</code>                         | Grupo de selección exclusiva segmentado (M3) |
| <code>CloseButton</code>                             | Botón de cerrar (back)                       |
| <code>BackButton</code>                              | Botón de retroceso                           |

## Controles de entrada

| Widget                                                                                   | Uso                                                                           |
|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <code>TextField</code>                                                                   | Campo de texto libre                                                          |
| <code>TextFormField</code>                                                               | Campo de texto con validación <code>Form</code>                               |
| <code>DropDownButtonFormField</code>                                                     | Selector desplegable (usa <code>initialValue</code> , no <code>value</code> ) |
| <code>DropDownMenu</code>                                                                | Menú desplegable M3 con filtro de texto                                       |
| <code>SearchBar</code>                                                                   | Barra de búsqueda M3 (con <code>SearchAnchor</code> )                         |
| <code>SearchAnchor</code>                                                                | Ancla para búsqueda con vista de resultados                                   |
| <code>MenuAnchor</code>                                                                  | Menú posicionado con ancla declarativa                                        |
| <code>Checkbox</code>                                                                    | Casilla de verificación                                                       |
| <code>Switch</code>                                                                      | Interruptor on/off (usa <code>activeThumbColor</code> )                       |
| <code>Radio</code>                                                                       | Botón de opción única (dentro de <code>RadioGroup</code> )                    |
| <code>RadioGroup</code>                                                                  | Agrupar <code>Radio</code> y maneja selección + teclado (3.32+)               |
| <code>Slider</code>                                                                      | Control deslizante                                                            |
| <code>RangeSlider</code>                                                                 | Control de rango                                                              |
| <code>DatePicker</code>                                                                  | Selector de fecha ( <code>showDatePicker</code> )                             |
| <code>TimePicker</code>                                                                  | Selector de hora ( <code>showTimePicker</code> )                              |
| <code>Autocomplete</code>                                                                | Autocompletado con sugerencias                                                |
| <code>SwitchListTile</code> / <code>CheckboxListTile</code> / <code>RadioListTile</code> | Controles con etiqueta en ListTile                                            |

## Selectores

| Widget        | Uso                                                                                                         |
|---------------|-------------------------------------------------------------------------------------------------------------|
| Chip          | Etiqueta simple                                                                                             |
| InputChip     | Chip interactivo seleccionable                                                                              |
| FilterChip    | Chip para filtrar (multiselección)                                                                          |
| ChoiceChip    | Chip de selección única                                                                                     |
| ActionChip    | Chip que dispara una acción                                                                                 |
| ToggleButtons | Botones de toggle múltiple                                                                                  |
| ExpansionTile | Fila expandible con contenido (usar <code>ExpansibleController</code> desde 3.32 para control programático) |

## Layout

| Widget       | Uso                                       |
|--------------|-------------------------------------------|
| Row          | Diseño horizontal                         |
| Column       | Diseño vertical                           |
| Flex         | Base de Row/Column con custom direction   |
| Expanded     | Ocupa espacio proporcional                |
| Flexible     | Ocupa espacio sin forzar tamaño mínimo    |
| Stack        | Superposición de widgets                  |
| Positioned   | Posición específica dentro de Stack       |
| IndexedStack | Stack con un hijo visible a la vez        |
| Wrap         | Envoltura automática (como flexbox wrap)  |
| CarouselView | Carrusel de tarjetas con snapping (3.35+) |

## Contenedores

| Widget    | Uso                                  |
|-----------|--------------------------------------|
| Container | Caja con decoración, padding, tamaño |
| SizedBox  | Caja de tamaño fijo                  |
| Padding   | Espaciado interno                    |

| Widget                                                                 | Uso                                           |
|------------------------------------------------------------------------|-----------------------------------------------|
| <code>Center</code>                                                    | Centra al hijo                                |
| <code>Align</code>                                                     | Alinea al hijo en una posición                |
| <code>AspectRatio</code>                                               | Mantiene relación de aspecto                  |
| <code>ConstrainedBox</code>                                            | Restricciones de tamaño                       |
| <code>FractionallySizedBox</code>                                      | Tamaño relativo al padre                      |
| <code>ColoredBox</code>                                                | Solo color (ligero)                           |
| <code>DecoratedBox</code>                                              | Solo decoración (ligero)                      |
| <code>Card</code>                                                      | Tarjeta elevada Material                      |
| <code>Card.filled</code> / <code>Card.outlined</code>                  | Variantes M3 sutiles                          |
| <code>Badge</code>                                                     | Indicador superpuesto (contador/notificación) |
| <code>ClipRect</code> / <code>ClipRRect</code> / <code>ClipOval</code> | Recorte visual                                |
| <code>SafeArea</code>                                                  | Evita áreas del sistema                       |
| <code>InkWell</code>                                                   | Área táctil con efecto Material               |
| <code>GestureDetector</code>                                           | Detección de gestos genérica                  |

## Scroll

| Widget                             | Uso                                  |
|------------------------------------|--------------------------------------|
| <code>ListView</code>              | Lista desplazable                    |
| <code>ListView.builder</code>      | Lista virtualizada                   |
| <code>ListView.separated</code>    | Lista con separadores                |
| <code>GridView</code>              | Cuadrícula desplazable               |
| <code>CustomScrollView</code>      | Scroll con slivers                   |
| <code>SingleChildScrollView</code> | Scroll para contenido único          |
| <code>AnimatedList</code>          | Lista con animaciones                |
| <code>ReorderableListView</code>   | Lista reordenable por drag           |
| <code>PageView</code>              | Páginas swipeables                   |
| <code>RefreshIndicator</code>      | Pull-to-refresh                      |
| <code>Scrollbar</code>             | Barra de scroll visible              |
| <code>NestedScrollView</code>      | Scroll anidado con header colapsable |

| Widget                            | Uso                              |
|-----------------------------------|----------------------------------|
| <code>NotificationListener</code> | Escucha notificaciones de scroll |

## Slivers

| Widget                           | Uso                               |
|----------------------------------|-----------------------------------|
| <code>SliverAppBar</code>        | AppBar que colapsa con scroll     |
| <code>SliverList</code>          | Lista dentro de CustomScrollView  |
| <code>SliverGrid</code>          | Grid dentro de CustomScrollView   |
| <code>SliverToBoxAdapter</code>  | Widget suelto en CustomScrollView |
| <code>SliverFillRemaining</code> | Ocupa el espacio restante         |
| <code>SliverPadding</code>       | Padding dentro de sliver          |
| <code>SliverAnimatedList</code>  | AnimatedList sliver               |

## Diálogos y notificaciones

| Widget                            | Uso                                  |
|-----------------------------------|--------------------------------------|
| <code>AlertDialog</code>          | Diálogo de alerta                    |
| <code>SimpleDialog</code>         | Diálogo simple                       |
| <code>AboutDialog</code>          | Diálogo Acerca de                    |
| <code>BottomSheet</code>          | Panel inferior                       |
| <code>SnackBar</code>             | Notificación temporal                |
| <code>Banner</code>               | Banner informativo en parte superior |
| <code>Tooltip</code>              | Tooltip en hover/longpress           |
| <code>PopupMenuButton</code>      | Menú contextual emergente            |
| <code>showDialog</code>           | Muestra un diálogo                   |
| <code>showModalBottomSheet</code> | Bottom sheet modal                   |
| <code>showMenu</code>             | Menú emergente posicionado           |

## Navegación

| Widget                                        | Uso                                                                  |
|-----------------------------------------------|----------------------------------------------------------------------|
| <code>Scaffold</code>                         | Estructura base de pantalla                                          |
| <code>AppBar</code>                           | Barra superior                                                       |
| <code>NavigationBar</code>                    | Barra de navegación inferior M3                                      |
| <code>NavigationDrawer</code>                 | Drawer lateral M3                                                    |
| <code>NavigationRail</code>                   | Barra lateral (tablet/desktop)                                       |
| <code>BottomNavigationBar</code>              | Barra inferior (legacy M2)                                           |
| <code>Drawer</code>                           | Drawer lateral (legacy M2)                                           |
| <code>TabBar</code> + <code>TabBarView</code> | Pestañas con vistas                                                  |
| <code>PopScope</code>                         | Intercepta botón de retroceso (reemplaza <code>WillPopScope</code> ) |
| <code>Hero</code>                             | Transición compartida entre pantallas                                |

## Datos y estado

| Widget                              | Uso                                                 |
|-------------------------------------|-----------------------------------------------------|
| <code>FutureBuilder</code>          | Widget reactivo a Future                            |
| <code>StreamBuilder</code>          | Widget reactivo a Stream                            |
| <code>ValueListenableBuilder</code> | Widget reactivo a <code>ValueNotifier</code>        |
| <code>ListenableBuilder</code>      | Widget reactivo a cualquier <code>Listenable</code> |
| <code>AnimatedBuilder</code>        | Widget reactivo a Animation                         |
| <code>LayoutBuilder</code>          | Widget reactivo a constraints del padre             |
| <code>OrientationBuilder</code>     | Widget reactivo a orientación                       |

## Decoración y efectos

| Widget                       | Uso                              |
|------------------------------|----------------------------------|
| <code>Divider</code>         | Línea divisoria horizontal       |
| <code>VerticalDivider</code> | Línea divisoria vertical         |
| <code>SizeBox.expand</code>  | Ocupa todo el espacio disponible |
| <code>Spacer</code>          | Espacio flexible en Row/Column   |

| Widget                       | Uso                                                                       |
|------------------------------|---------------------------------------------------------------------------|
| <code>IntrinsicHeight</code> | Altura intrínseca (mide hijos)                                            |
| <code>IntrinsicWidth</code>  | Ancho intrínseco (mide hijos)                                             |
| <code>Theme</code>           | Aplica tema a hijos                                                       |
| <code>MediaQuery</code>      | Información de pantalla (usa <code>sizeOf</code> / <code>widthOf</code> ) |
| <code>Overlay</code>         | Superposición de capas                                                    |

## Animaciones implícitas

| Widget                                | Uso                                    |
|---------------------------------------|----------------------------------------|
| <code>AnimatedContainer</code>        | Container con animación de propiedades |
| <code>AnimatedOpacity</code>          | Fade in/out animado                    |
| <code>AnimatedPadding</code>          | Padding animado                        |
| <code>AnimatedPositioned</code>       | Positioned animado                     |
| <code>AnimatedAlign</code>            | Alineación animada                     |
| <code>AnimatedSize</code>             | Tamaño animado                         |
| <code>AnimatedSwitcher</code>         | Transición entre widgets hijos         |
| <code>AnimatedDefaultTextStyle</code> | Estilo de texto animado                |
| <code>AnimatedRotation</code>         | Rotación animada                       |
| <code>AnimatedScale</code>            | Escala animada                         |
| <code>AnimatedSlide</code>            | Desplazamiento animado                 |
| <code>TweenAnimationBuilder</code>    | Animación tween declarativa            |

## Animaciones explícitas

| Widget                           | Uso                         |
|----------------------------------|-----------------------------|
| <code>AnimationController</code> | Controlador de animación    |
| <code>Animation</code>           | Valor animado               |
| <code>Tween</code>               | Interpolación entre valores |
| <code>CurvedAnimation</code>     | Curva de easing             |
| <code>FadeTransition</code>      | Transición de opacidad      |

| Widget                          | Uso                               |
|---------------------------------|-----------------------------------|
| <code>ScaleTransition</code>    | Transición de escala              |
| <code>SizeTransition</code>     | Transición de tamaño              |
| <code>SlideTransition</code>    | Transición de desplazamiento      |
| <code>RotationTransition</code> | Transición de rotación            |
| <code>AnimatedBuilder</code>    | Builder genérico de animaciones   |
| <code>Ticker</code>             | Tick de sincronización            |
| <code>CustomPainter</code>      | Dibujo vectorial personalizado    |
| <code>Transform</code>          | Transformación 2D/3D (no animada) |

## Material 3 específicos

| Widget                                                | Uso                            |
|-------------------------------------------------------|--------------------------------|
| <code>NavigationBar</code>                            | Barra inferior Material 3      |
| <code>NavigationDrawer</code>                         | Drawer Material 3              |
| <code>NavigationRail</code>                           | Barra lateral (tablet/desktop) |
| <code>SegmentedButton</code>                          | Selección exclusiva segmentada |
| <code>DropDownMenu</code>                             | Menú desplegable con búsqueda  |
| <code>SearchBar</code> + <code>SearchAnchor</code>    | Búsqueda M3                    |
| <code>Card.filled</code> / <code>Card.outlined</code> | Tarjetas M3 sutiles            |
| <code>Badge</code>                                    | Indicador superpuesto          |
| <code>CarouselView</code>                             | Carrusel M3 (3.35+)            |

## Estructurales (app-level)

| Widget                          | Uso                   |
|---------------------------------|-----------------------|
| <code>MaterialApp</code>        | App con tema Material |
| <code>MaterialApp.router</code> | App con GoRouter      |
| <code>Scaffold</code>           | Pantalla completa     |
| <code>ScaffoldMessenger</code>  | Muestra SnackBars     |
| <code>Navigator</code>          | Pila de navegación    |

| Widget                  | Uso                   |
|-------------------------|-----------------------|
| <code>Router</code>     | Enrutador declarativo |
| <code>WidgetsApp</code> | App base sin Material |

## Widgets Cupertino (iOS-style)

| Widget                                  | Uso                    |
|-----------------------------------------|------------------------|
| <code>CupertinoPageScaffold</code>      | Página iOS             |
| <code>CupertinoNavigationBar</code>     | Barra superior iOS     |
| <code>CupertinoTabBar</code>            | Barra de tabs iOS      |
| <code>CupertinoButton</code>            | Botón iOS              |
| <code>CupertinoTextField</code>         | Campo de texto iOS     |
| <code>CupertinoSwitch</code>            | Switch iOS             |
| <code>CupertinoSlider</code>            | Slider iOS             |
| <code>CupertinoActivityIndicator</code> | Indicador de carga iOS |
| <code>CupertinoAlertDialog</code>       | Alerta iOS             |
| <code>CupertinoDatePicker</code>        | Date picker iOS        |
| <code>CupertinoTimerPicker</code>       | Timer picker iOS       |

Nota: No es necesario usar Cupertino widgets. Puedes usar Material 3 con platform adaptivity.

## Referencias

- Flutter | [Widget catalog](#) — Catálogo completo de widgets por categoría
- Flutter | [Material 3](#) — Componentes y estilos M3
- Flutter | [API reference](#) — Documentación de la API de Flutter
- Flutter | [Layouts](#) — Guía de layouts en Flutter

## Lo que sigue

Este módulo terminó. Continúa con [16-BLOC-CUBIT](#) para aprender a manejar el estado de tu aplicación con BLoC y Cubit.